



BOOTCAMP FRONTEND & REACT · SEMANA 4

La tienda terminada

Clase 11 — respaldo, filtros, buscador, carrito con cantidades y checkout:
TechCart queda hecha una tienda de verdad

Último día de JavaScript

Proyecto TechCart

STEERCLAVE

TENER DATOS DE VERDAD DEJÓ TRES HUECOS A LA VISTA · HOY SE CIERRAN

Agenda

1. El respaldo

que la tienda abra aunque la red no esté

3. El buscador

texto, tildes y los dos filtros a la vez

5. Carrito con cantidades

una línea por producto, con más y menos

7. El checkout

del carrito al pedido, con su comprobante

2. Filtro por categoría

y la idea del día: lo que se ve se calcula

4. El estado vacío

y el texto que escribe una persona

6. El carrito guardado de ayer

la aduana que evita el NaN

8. Cierre

errores frecuentes · resumen de JavaScript · tareas



Receso de 15 min después del **filtro por categoría**, antes del buscador

Tres por el chat

¿Qué devuelve `fetch`: los datos o una promesa?

Si el servidor responde `404`, ¿se dispara el `catch`?

¿Para qué sirve el adaptador de `api.js`?

CON SEIS PRODUCTOS ESCRITOS A MANO ESTO NO SE NOTABA

Lo que le falta a TechCart

38 en una lista

para llegar a las tablets hay que pasar por dieciséis teléfonos. **No hay filtro ni buscador.**

3 filas iguales

el mismo producto comprado tres veces ocupa tres filas. **Ninguna tienda hace eso.**

no se compra

el formulario está en la página desde la **Clase 1** y nunca armó un pedido.

Y un cuarto hueco que no se ve: con la red caída la tienda queda **en blanco**. El aviso está bien puesto y el botón de reintentar también. Pero una página vacía no se lee como "la red falló": se lee como **"esto está roto"**, y la gente se va.

LA PREGUNTA NO ES SI LA RED VA A FALLAR, ES QUÉ HACE LA TIENDA CUANDO FALLA

Degradación elegante

hoy: la tienda en blanco

aviso rojo, cero productos. Quien venía a comprar no distingue "la red falló" de "esto está roto", y se va.

desde hoy: se cae menos

los 38 productos de la última carga buena, y arriba un cartel que dice **de cuándo** son. Se puede mirar, filtrar y llenar el carrito.

Es lo que hacen las aplicaciones que usan todos los días: la música sin conexión, el correo que deja leer lo que ya bajó. No les dan todo, pero tampoco les dan una pantalla vacía. La frase para llevarse: **cuando algo falla, no se cae todo: se cae menos.**

HAY UNA RESPUESTA OBVIA, Y ES LA MALA

De dónde sale la copia

ESCRIBIRLA A MANO

```
// datos.js
export const productos = [
  { nombre: "MacBook", precio: 1999
  },
  ...
];
```

dos verdades sobre el mismo catálogo. Cambia un precio allá y aquí sigue el viejo. Gana un campo el adaptador y hay que acordarse. **Nadie se acuerda.**



GUARDAR LO QUE LLEGÓ

```
// respaldo.js
{ fecha: "2026-10-09T17:58...",
  productos: [ /* los 38 */ ] }
```

sale de la propia API, **ya traducida por el adaptador**, así que tiene exactamente la misma forma. Se llama **instantánea** (*snapshot*).

Y siempre con fecha. Una copia sin fecha es peor que no tenerla: la tienda serviría datos de hace seis meses con cara de datos frescos. La fecha es lo que permite **decirle a la persona de cuándo** es lo que está viendo.

QUIEN SABE DE DÓNDE SALEN LOS DATOS ES LA CAPA DE DATOS, NO LA PANTALLA

api.js decide entre la red y la copia

```
export const obtenerProductos = async () => {  
  try {  
    const listas = await Promise.all(categorias.map(pedirCategoria));  
    const productos = listas.flat();  
    guardarRespaldo(productos); // ← la copia sale de aquí, ya traducida  
    return { productos, origen: "api", fecha: null };  
  } catch (error) {  
    const copia = leerRespaldo();  
    if (!copia) throw error; // ← sin copia, vuelve el error de la Clase 10  
    return { productos: copia.productos, origen: "respaldo", fecha: copia.fecha };  
  }  
};
```

dos cosas que contar

los productos y de dónde salieron → se devuelve un objeto

guardar aquí adentro

lo guardado ya pasó por el adaptador: misma forma, siempre

throw error

sin copia no se inventa nada: lista vacía diría una mentira

origen es texto

"api" / "respaldo" se leen solos y admiten un tercero

SERVIR LA COPIA Y NO DECIRLO ES MENTIRLE A QUIEN COMPRA

Una línea cubre los **dos** casos

```
avisoRespaldo.hidden = !esRespaldo;
```

origen: "api"

el cartel se esconde solo. Nadie tiene que acordarse de apagarlo en otro sitio.

origen: "respaldo"

"No pudimos conectarnos con la tienda. Estás viendo el catálogo guardado el **9 de octubre de 2026 a las 5:58 p. m.**, así que los precios y el stock pueden haber cambiado."

no hay un if que enciende y otro que apaga: hay un dato y la pantalla lo refleja

SI HOY SE OLVIDAN DE TODO LO DEMÁS Y SE ACUERDAN DE ESTE DIBUJO, LA CLASE VALIÓ

Lo que se ve no se guarda: se **calcula**

catalogo

todo lo que llegó.
No se toca más.

categoriaActiva

qué botón
está apretado

termino

qué hay escrito
en el buscador



```
const productosVisibles = () => buscar(filtrarPorCategoria(catalogo, categoriaActiva),  
                                         termino);
```

~~let~~ productosFiltrados

dos listas que tienen que coincidir. Cada cambio hay que acordarse de recalcularla. El día que falte en un sitio: la grilla dice 3 y la línea dice 38.

estado derivado

un dato calculado **no puede estar desactualizado**. Ese bug no se puede ni escribir, porque no hay nada que desincronizar.

UN DETALLE DE OFICIO QUE SEPARA AL QUE PROGRAMA DEL QUE COPIA

La **etiqueta** es para la persona. El **valor**, para la máquina.

LO QUE LEE QUIEN COMPRA

Laptops
Teléfonos
Tablets
Accesorios

≠

LO QUE USA EL CÓDIGO

```
"laptops"  
"smartphones"  
"tablets"  
"mobile-accessories"
```

El día que la tienda quiera que ese botón diga "Accesorios y audio", se cambia el **texto** y el código que filtra **no se entera**. La regla: **nunca decidan la lógica leyendo el texto de la pantalla**. Vive en `datos.js`, con las categorías, porque también es una decisión de la tienda.

ESCRIBIRLOS A MANO SERÍA TENER DOS LISTAS DE CATEGORÍAS EN EL PROYECTO

Los botones salen del **dato**

```
const botones = [  
  botonCategoria(TODAS, "Todas", catalogo.length, categoriaActiva === TODAS),  
  ...categorias.map((categoria) =>  
    botonCategoria(categoria, etiquetaCategoria[categoria],  
      cuenta[categoria] ?? 0, categoria === categoriaActiva)),  
];
```

Todas 38 · Laptops 5 · Teléfonos 16 · Tablets 3 · Accesorios 14

<button>, no <a>

no lleva a otro sitio: cambia lo que se ve en este. Y no hay salto que cancelar

aria-pressed

el color y lo que anuncia el lector salen del mismo dato

contarPorCategoria

escrita en la Clase 8 y sin tocar desde entonces

un listener

delegado en el contenedor: los botones se redibujan en cada clic

filter nunca muta. Por eso volver a "Todas" devuelve los 38 intactos. Si el filtro mutara, cada clic se comería productos y no habría forma de recuperarlos sin volver a pedirlos.

LO QUE HACE QUE UNA PANTALLA SE SIENTA VIVA

El evento **input**: en cada tecla

input

cada vez que el **valor cambia**:
teclear, pegar con el ratón, dictar
por voz, autocompletar

keydown

cada **tecla**. No cubre pegar ni
dictar, que es justo lo que
queremos cubrir

change

en un campo de texto, **una sola vez**, al salir del campo. Sirve para
un `<select>`

```
campoBuscar.addEventListener("input", (evento) => {  
  termino = evento.target.value;  
  pintarCatalogo();  
});
```

Este buscador no le pide nada al servidor: filtra los 38 que ya están en memoria, y por eso puede reaccionar en cada tecla sin ningún costo. El día que le pregunte a la API, diez letras serían diez peticiones y harían falta dos cosas: el **rebote** (*debounce*) y **cancelar** la petición anterior.

DOS TEXTOS QUE UNA PERSONA LEE IGUALES Y EL PROGRAMA NO

Mayúsculas y tildes

```
"Teléfonos".toLowerCase().includes("telefonos") // → false
```

```
export const normalizar = (texto) =>
  texto
    .normalize("NFD") // "é" pasa a ser "e" + la tilde suelta
    .replace(/[\u0300-\u036f]/g, "") // se borran todas las tildes sueltas
    .toLowerCase();
```

de los dos lados

se normaliza lo que se escribe y lo que se busca

trim()

un espacio suelto no debería vaciar el catálogo

texto vacío

devuelve la lista entera: no buscar nada no es buscar "nada"

nombre · marca · etiqueta

los productos vienen en inglés: "telefonos" encuentra por la etiqueta

EL MISMO HUECO EN LA PANTALLA, DOS CAUSAS DISTINTAS, DOS MENSAJES DISTINTOS

Cuando los filtros no dejan **nada**

CLASE 10 • EL SERVIDOR CONTESTÓ VACÍO

No hay productos para mostrar.

CLASE 11 • TUS FILTROS NO DEJARON NINGUNO

No encontramos nada para "zapatos".
Prueba con otra palabra o quita el filtro.

innerHTML con el término

interpreta lo que recibe. Una etiqueta escrita en el buscador se ejecuta dentro de la tienda, con acceso al carrito y a todo lo guardado. Se llama **XSS**.

createElement + textContent

trata el valor como **texto** y nunca como etiquetas. No hace falta ninguna lista de caracteres prohibidos, que siempre se queda corta.

la regla, de una línea: si un texto lo escribió una persona, nunca va crudo en innerHTML

EL CAMBIO MÁS GRANDE DEL DÍA, Y ES UNO SOLO

Un ítem ya no es un producto: es un producto con una **cantidad**

hoy

MacBook Pro 14 · \$1999.99
MacBook Pro 14 · \$1999.99
MacBook Pro 14 · \$1999.99

desde hoy

MacBook Pro 14 – 3 + **\$5999.97** Quitar

agregar

busca si ya está; si está, sube la cantidad

quitar vuelve al id

ya no hay ids repetidos, así que el filtro por id acierta

bajar a 0 = quitar

un filter al final, y el botón de menos no necesita ningún if

unidades, no filas

el resumen suma cantidades y el subtotal multiplica

no arreglamos "quitar": arreglamos la forma de los datos, y "quitar" se arregló solo

Una **copia** con un campo más

```
const sumarLinea = (lineas, producto, cuantas) => {  
  const existente = lineas.find(({ id }) => id === producto.id);  
  
  if (!existente) {  
    return [...lineas, { ...producto, cantidad: cuantas }]; // no estaba  
  }  
  
  return lineas.map((linea) =>  
    linea.id === producto.id ? { ...linea, cantidad: linea.cantidad + cuantas } : linea);  
};
```

existente.cantidad++

funcionaría hoy. En React **no**: no vuelve a pintar porque el arreglo sigue siendo el mismo objeto de antes. Y sin copia no se puede comparar el antes con el después.

map + ternario

se recorre la lista entera y se devuelve, por cada línea, o una **copia modificada** o **la misma tal cual**. Arreglo nuevo donde cambió exactamente una.

UN BOTÓN MÁS ES UN IF MÁS, NO UN LISTENER MÁS

Un solo **listener**, tres botones

```
listaCarrito.addEventListener("click", (evento) => {  
  const boton = evento.target.closest("button[data-accion]"); // ← cualquier acción  
  if (!boton) return;  
  
  const id = Number(boton.dataset.id); // dataset SIEMPRE devuelve texto  
  const accion = boton.dataset.accion;  
  
  if (accion === "mas") carrito = cambiarCantidad(carrito, id, 1);  
  if (accion === "menos") carrito = cambiarCantidad(carrito, id, -1);  
  if (accion === "quitar") carrito = quitarDelCarrito(carrito, id);  
  
  guardarCarrito(carrito);  
  pintarCarrito();  
});
```

CLASE 9

`closest("button[data-accion='quitar']")`

CLASE 11

`closest("button[data-accion]")`

LO ESCRIBIÓ LA VERSIÓN ANTERIOR DE NUESTRO PROPIO PROGRAMA, HACE MEDIA HORA

El carrito guardado dice \$NaN

LO QUE HAY EN EL NAVEGADOR

```
[
  { id: 6, precio: 1999.99 },
  { id: 6, precio: 1999.99 }
]
// mismo id repetido · sin cantidad
```

```
1999.99 * undefined // → NaN
0 + NaN // → NaN
Total: $NaN
```



LA ADUANA DE LEERCARRITO

```
const agruparLineas = (guardado) =>
  guardado.reduce((lineas, linea) =>
    sumarLinea(
      lineas, linea,
      linea.cantidad ?? 1), []);
```

La **misma pieza** que usa el botón de agregar. Tres líneas sueltas entran y sale una con cantidad 3.

Se llama migración de datos y es uno de los bugs más caros que existen: le pasa solo a quien ya tenía datos guardados, y **no se reproduce en la máquina de quien programó**, porque probó con el almacenamiento limpio. Nos pasó en media hora, en una tienda de juguete. La otra cura, para cambios grandes: **versionar la clave** (techcart_carrito_v2).

ONCE CLASES DESPUÉS, EL FORMULARIO DE LA CLASE 1 VA A HACER LO QUE PROMETÍA

La validación que **no escribimos** ya está trabajando

1 • submit

y preventDefault, o la página recarga

2 • ¿es válido?

checkValidity + reportValidity

3 • leer

FormData + Object.fromEntries

4 • armar y mostrar

el pedido y su comprobante

5 • vaciar

en memoria y en localStorage

lo que regaló la Clase 1

required, type="email", minlength, pattern. Mensajes en castellano puestos por el navegador, accesibles, y funcionan hasta sin JavaScript. Once clases después ahorran cincuenta if.

pero no es seguridad

cualquiera abre DevTools, borra el required y manda el formulario vacío. **Validar en el cliente para ayudar a la persona, y otra vez en el servidor, que es el único sitio donde la persona no manda.**

y el orden: primero se muestra el comprobante, después se vacía el carrito, porque el paso que no se puede perder va primero

EL FORMULARIO ENTERO, EN UN OBJETO, EN UNA LÍNEA

FormData + Object.fromEntries

```
const datos = Object.fromEntries(new FormData(formularioCompra));
```

FORMDATA DA PARES

```
["nombre", "Ana Pérez"]  
["email", "ana@example.com"]  
["pago", "tarjeta"]
```



FROMENTRIES DA UN OBJETO

```
{  
  nombre: "Ana Pérez",  
  email: "ana@example.com",  
  pago: "tarjeta"  
}
```

del atributo name

no del id. Eso viene de la Clase
1 y hoy se entiende por qué

los radios, solos

trae el que está marcado, sin un
if

todo es texto

incluso fechas y números. Si
hace falta un número,
Number(...)

las cuentas no se repiten

salen de resumenCarrito, no se
vuelven a hacer aquí

HACE CINCO SEMANAS ESTO ERA UN ARCHIVO HTML VACÍO

Ocho archivos, **un trabajo** cada uno

datos.js

qué vende la tienda y cómo se llama cada categoría

api.js

habla con el servidor, traduce y decide entre red y copia

respaldo.js

la copia del catálogo, con su fecha

tienda.js

cuentas, filtro y búsqueda. No sabe de dónde salen los productos

carrito.js

las reglas del carrito y dónde se guarda

formato.js

cómo se muestra un precio y una fecha

ui.js

cómo se ve una cosa

main.js

orquesta y escucha. El único que toca el DOM

un dato cambia

en el listener,
una línea

**se manda a pintar**

la función que
dibuja esa zona

**la pantalla lo
refleja**

entera, no
a pedacitos

CASI NINGUNO DA UN ERROR ROJO EN LA CONSOLA · ESE ES JUSTO EL PROBLEMA DE HOY

Errores frecuentes

`base.sort()` → base queda ordenado · `orden === base` es true

1 • filtrar u ordenar **mutando** el arreglo original: volver a "Todas" ya no trae los 38. Ordena una copia: `[...lista].sort()`

`"Teléfonos".toLowerCase().includes("telefonos")` → false

2 • comparar texto **sin normalizar**. El buscador no encuentra lo que está ahí, y no hay error

`TypeError: Cannot read properties of undefined (reading 'nombre')`

3 • olvidar que lo que sale del HTML es **cadena**: `dataset.id` da "78". También `"2" + 1` → "21" y `"10" < "9"` → true

(sin error: la fila dice una cosa y el total otra)

4 • calcular el carrito en un sitio y olvidarlo en otro. Una función hace la cuenta y los demás la llaman

(sin error: mañana vuelve lo que ya se compró)

5 • confirmar el pedido y **no vaciar** el carrito. O vaciarlo solo en memoria y no en `localStorage`

(sin error: precios de hace tres meses con cara de nuevos)

6 • guardar el respaldo **sin fecha**. Se guarda con fecha y la fecha se muestra

Total: \$NaN

7 • el carrito guardado con la forma vieja: `1999.99` * `undefined` → NaN, y el veneno se propaga sin avisar

CINCO CLASES DE JAVASCRIPT, Y CADA UNA SE APOYA EN LA ANTERIOR

Todo JavaScript, en una pantalla

C7 · map filter reduce

transformar, elegir y resumir
una lista sin un solo for

C8 · ES6+

desestructuración, template
literals, spread, copiar en vez
de mutar

C8 · módulos

un archivo, un trabajo; import y
export

C9 · el DOM

querySelector, textContent
frente a innerHTML, classList,
dataset

C9 · eventos

delegación, preventDefault,
checkValidity

C9 · localStorage

el carrito sobrevive al refresco

C10 · fetch y promesas

un 404 NO rechaza: el throw es
nuestro

C10 · async/await

y Promise.all para lo que no
depende de nada más

C10 · el adaptador

un solo archivo habla el idioma
de la API

C11 · el respaldo

una instantánea con fecha, y un
aviso honesto

C11 · estado derivado

tres datos y una función que
calcula lo que se ve

C11 · el MVP

carrito con cantidades y
checkout: se compra de verdad

y la única frase que hay que llevarse: hay datos, y hay funciones que dibujan esos datos

5 tareas, en tres niveles

FÁCIL

Replicar la clase y agregar una quinta categoría. La tienda como quedó hoy, probada también con la red en Offline. Más: mens-watches en categorias y "Relojes" en etiquetaCategoria. **Dos líneas de datos y cero de código.**

INTERM. 1

Que no se pueda comprar más de lo que hay. El tope de stock en cambiarCantidad y en sumarLinea. Y si sobra tiempo, deshabilitar el botón + al llegar al tope: hoy la tienda se defiende pero no habla.

INTERM. 2

Ordenar el catálogo. Un <select> con tres opciones y el evento change. Es un **cuarto dato** que entra en productosVisibles, al final. Ojo: sort muta, va sobre una copia.

DIFÍCIL 1

El filtro y la búsqueda en la dirección. ?categoria=tablets&q=pro, leída al abrir y actualizada en cada cambio, con URLSearchParams e history.replaceState. Aquí sí hace falta el **rebote**, o el historial se llena.

DIFÍCIL 2

"Mis pedidos". Guardar cada pedido en techcart_pedidos (leer, copiar con spread, guardar) y darle su pantalla con <details>. El nombre lo escribió una persona: va con textContent.

¿Dónde está escrito **qué es un producto?**

Hoy pasó **tres veces** lo mismo y nadie avisó: el carrito guardado sin cantidad iba a pintar \$NaN; el buscador se habría roto si marca llegara vacía; y `dataset.id` es texto, no número. Los tres son el mismo error: **un dato que no tenía la forma que el código esperaba.**

La respuesta está en **mapearProducto**, en **tarjetaProducto**, en el buscador y en mi cabeza. Nadie los vigila.

La solución se escribe **una vez**, en un sitio, y el editor avisa **antes de correr el programa**. Se llama **TypeScript**.

Traigan la tienda funcionando y las cinco tareas intentadas. Nos vemos.